

Practical 5

AIM : Support Vector Machines (SVM)

1. Implement the SVM algorithm for binary classification.
2. Train an SVM model using a given dataset and optimize its parameters.
3. Evaluate the performance of the SVM model on test data and analyze the results.

CODE :

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
ConfusionMatrixDisplay

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

print("Dataset Shape:", X.shape)
print("Classes:", data.target_names)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Optimize SVM parameters using GridSearchCV
parameters = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 0.01, 0.1, 1],
    'kernel': ['rbf']
}

grid = GridSearchCV(
    SVC(),
```

```
parameters,
cv=5,
scoring='accuracy'
)

grid.fit(X_train, y_train)

# Best SVM model
model = grid.best_estimator_

# Predictions
y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)

print("\nSupport Vector Machine (SVM)")
print("-----")
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))

print("\nBest Parameters:")
print(grid.best_params_)

print("\nActual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:", accuracy * 100, "%")

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=data.target_names
))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=data.target_names
)

disp.plot()
plt.title("SVM Confusion Matrix")
plt.tight_layout()
plt.show()
```

OUTPUT :

```

File Edit Selection View Go Run ... AI Practicals
Problems Output Debug Console Terminal Ports
python Update
AI Practicals
  Practical3.py
  Practical4.py
  Practical5.py
PS C:\Users\DELL\OneDrive\Desktop\AI Practicals> python Practical5.py
Support Vector Machine (SVM)
-----
Training Samples: 455
Testing Samples: 114

Best Parameters:
{'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}

Actual Values:
[0 1 0 1 0 1 1 0 0 0 1 0 1 0 0 1 1 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0 0 1
 1 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 0 1 1 1 1 0 0 1 1 1 1 1 1 1 1 1 1 1 0
 0 0 1 1 1 1 1 0 1 0 1 1 1 1 1 1 1 0 0 0 1 0 1 0 0 0 1 0 0 1 0 1 0 1
 0 1 1]

Predicted Values:
[0 1 0 1 0 1 1 0 0 0 1 0 1 0 0 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0 0 1
 1 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0
 0 0 1 1 1 1 1 0 1 0 1 1 1 1 1 1 1 0 0 0 1 0 1 0 0 0 1 0 0 1 0 1 0 1
 0 1 1]

Accuracy: 98.24561403508771 %

Classification Report:
      precision    recall  f1-score   support

 malignant      0.98      0.98      0.98        42
   benign      0.99      0.99      0.99        72

 accuracy      0.98      0.98      0.98       114
 macro avg      0.98      0.98      0.98       114
 weighted avg      0.98      0.98      0.98       114
  
```

confusion matrix Graph :

